

VIVA Q&A PREPARATION GUIDE

High Performance Computing (HPC)
&
Deep Learning (DL)

Student:	Manas Suhas Ohal
Roll No.:	75 BE-B
Branch:	Computer Engineering
University:	Savitribai Phule Pune University
Subject 1:	HPC — BFS, DFS, Bubble Sort, Merge Sort, Reduction
Subject 2:	DL — Boston Housing, IMDB Sentiment, CNN Fashion

Covers all practicals with examiner-level questions and detailed answers

TABLE OF CONTENTS

SECTION A — HIGH PERFORMANCE COMPUTING (HPC)

- Practical 1 — BFS (Breadth First Search) ([bfs.py](#))
- Practical 2 — Parallel DFS (Depth First Search) ([dfs.py](#))
- Practical 3 — Parallel Bubble Sort ([bubble.py](#))
- Practical 4 — Parallel Merge Sort ([merge.py](#))
- Practical 5 — Parallel Reduction ([reduction.py](#))

SECTION B — DEEP LEARNING (DL)

- Practical 1 — Boston Housing Price Prediction ([DL1_M.pdf](#))
- Practical 2 — IMDB Movie Sentiment Analysis ([DL2_M.pdf](#))
- Practical 3 — Fashion MNIST CNN Classifier ([DL3_M.pdf](#))

HPC PRACTICAL 1 — BFS (BREADTH FIRST SEARCH)

Aim: Implement BFS graph traversal

File: bfs.py | Data Structure: Queue (deque) | Graph: Undirected

— BASIC QUESTIONS —

Q1. What is BFS? Explain in simple terms.

BFS (Breadth First Search) is a graph/tree traversal algorithm that explores all neighbors of a node before moving to the next level. It uses a queue (FIFO) to keep track of nodes to visit.

Think of it like ripples in water — it spreads level by level from the starting point. BFS visits all nodes at distance 1 first, then distance 2, and so on — like exploring floor by floor in a building.

Q2. What data structure is used in BFS and why?

A queue (FIFO — First In First Out) is used. Python's deque from the collections module is preferred because deque.popleft() is $O(1)$, whereas a regular list's pop(0) is $O(n)$.

```
from collections import deque
queue = deque()
queue.append(node) #  $O(1)$  — add to back
node = queue.popleft() #  $O(1)$  — remove from front
```

Q3. What is the time and space complexity of BFS?

Time Complexity: $O(V + E)$ — V = number of vertices (each visited once), E = number of edges (each edge checked once).

Space Complexity: $O(V)$ — for the visited[] array and the queue, which can hold at most V nodes in the worst case.

Q4. What is an adjacency list? Why not use an adjacency matrix?

An adjacency list is a list of lists where adj[u] stores all neighbors of node u.

```
self.adj = [[] for _ in range(vertices)]
self.adj[u].append(v)
self.adj[v].append(u) # undirected
```

Adjacency matrix is $O(V^2)$ space. For sparse graphs (few edges), adjacency list uses only $O(V+E)$ memory — much more efficient. BFS only needs to iterate over actual neighbors, not all V nodes.

Q5. What is the difference between directed and undirected graph? Which does bfs.py use?

In an undirected graph, edges go both ways — if u connects to v, then v also connects to u. In a directed graph, edges have direction — $u \rightarrow v$ does not mean $v \rightarrow u$.

bfs.py uses an undirected graph — `add_edge()` adds `v` to `adj[u]` AND `u` to `adj[v]` (both directions).

— ADVANCED QUESTIONS —

Q6. What are the applications of BFS?

1. Shortest path in unweighted graphs — BFS guarantees the shortest path. 2. Social networks — finding friends of friends (LinkedIn connections). 3. Web crawlers — crawling pages level by level. 4. GPS navigation — finding nearest fuel stations, hospitals. 5. Peer-to-peer networks — finding closest peer. 6. Garbage collection in programming languages.

Q7. What happens if the graph is disconnected? Will BFS cover all nodes?

No — this BFS implementation only traverses the connected component containing the start node. Disconnected nodes won't be visited.

To handle disconnected graphs, loop over all vertices and call BFS for each unvisited node:

```
for i in range(V):
    if not visited[i]:
        g.bfs(i)
```

Q8. Can BFS detect cycles in a graph?

Yes. During BFS, if we encounter a neighbor that is already visited AND is not the parent of the current node, then a cycle exists.

Tip: DFS is more commonly used for cycle detection, but BFS can also do it.

HPC PRACTICAL 2 — PARALLEL DFS (DEPTH FIRST SEARCH)

Aim: Implement Parallel DFS using ThreadPoolExecutor

File: dfs.py | Method: Recursion + concurrent.futures | Graph: Undirected

— BASIC QUESTIONS —

Q1. What is DFS? How is it different from BFS?

DFS (Depth First Search) explores as deep as possible along one branch before backtracking. It uses a stack (or recursion).

BFS vs DFS: — BFS: queue, level-by-level, finds shortest path. — DFS: stack/recursion, goes deep first, good for cycle detection, topological sort. — BFS uses more memory for wide graphs; DFS uses more for deep graphs. — BFS guarantees shortest path in unweighted graphs; DFS does not.

Q2. How is parallelism achieved in the DFS code?

ThreadPoolExecutor is used to visit all neighbors of the current node in parallel via `executor.map()`. Each thread recursively calls `parallel_dfs` on its assigned neighbor.

```
def visit(neighbor):
    self.parallel_dfs(neighbor)

with ThreadPoolExecutor() as executor:
    executor.map(visit, self.adj[node])
```

Q3. What is ThreadPoolExecutor? Difference between `executor.map()` and `executor.submit()`?

ThreadPoolExecutor manages a pool of worker threads automatically. You don't create or destroy threads manually.

`executor.submit(fn, arg)` — submits a single task, returns a Future. Call `.result()` to get the value.
`executor.map(fn, iterable)` — applies function to each item in parallel. Simpler for batch operations.

```
f = executor.submit(my_func, arg1) # single task
result = f.result() # block till done

results = executor.map(my_func, [1,2,3]) # 3 parallel tasks
```

Q4. Does Python achieve true parallelism with threads? Why or why not?

No. Python has the GIL (Global Interpreter Lock) — only one thread can execute Python bytecode at a time.

For CPU-bound tasks (sorting, graph traversal): threads give no speedup because of the GIL. Use multiprocessing module instead. For I/O-bound tasks (network calls, file reads): threads work well — the GIL is released during I/O waits.

Tip: Say in exam: 'This code demonstrates the concept of parallel DFS, but true CPU parallelism in Python requires multiprocessing due to the GIL.'

Q5. What is a race condition? Does the DFS code have one?

A race condition occurs when two threads read/write shared data simultaneously, producing unpredictable results.

Important: Yes, this DFS code potentially has a race condition on the `visited[]` array. Two threads could simultaneously check `visited[node]==False` for the same node and both proceed — printing a node twice. Fix: Use `threading.Lock()` around the visited check-and-set.

— ADVANCED QUESTIONS —

Q6. What are the applications of DFS?

1. Cycle detection in graphs. 2. Topological sorting (scheduling tasks with dependencies). 3. Maze solving — explores one path fully before backtracking. 4. Connected components detection. 5. Strongly Connected Components (Tarjan's, Kosaraju's algorithms). 6. Path finding in games/AI. 7. Expression tree evaluation.

Q7. What is the time and space complexity of DFS?

Time: $O(V + E)$ — same as BFS. Each vertex visited once, each edge checked once.

Space: $O(V)$ — for the visited array, plus the recursion call stack which can go $O(V)$ deep in the worst case (linear graph).

Tip: Python's default recursion limit is 1000. Very deep graphs can cause `RecursionError`. Use `sys.setrecursionlimit(n)` to increase it.

HPC PRACTICAL 3 — PARALLEL BUBBLE SORT

Aim: Implement Sequential and Parallel Bubble Sort with timing comparison

File: `bubble.py` | Method: Odd-Even Transposition | `ThreadPoolExecutor`

— BASIC QUESTIONS —

Q1. How does bubble sort work? Explain with an example.

Bubble sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element 'bubbles up' to its correct position.

```
Array: [5, 3, 8, 1]
Pass 1: [3,5,8,1] → [3,5,1,8] (8 bubbles to end)
Pass 2: [3,1,5,8] (5 moves right)
Pass 3: [1,3,5,8] (sorted)
```

Time: $O(n^2)$. Space: $O(1)$ — in-place sorting.

Q2. How does parallel bubble sort work? What is odd-even transposition?

Odd-even transposition alternates between two phases: Even phase: Compare pairs at indices (0,1), (2,3), (4,5)... simultaneously. Odd phase: Compare pairs at indices (1,2), (3,4), (5,6)... simultaneously.

Pairs within each phase are independent — no two pairs share an index — so they can be compared in parallel without race conditions.

```
start = i % 2 # 0 for even pass, 1 for odd pass
executor.map(compare_swap, range(start, n-1, 2))
```

Q3. Why don't we need locks in parallel bubble sort?

Because in each phase, the pairs being compared are non-overlapping in indices. For example in even phase: pair (0,1) and pair (2,3) don't share any index. So two threads can never write to the same array position simultaneously — no race condition, no locks needed.

Key insight: careful index design eliminates the need for synchronization.

Q4. Is bubble sort stable? What does 'stable' mean?

Yes, bubble sort is stable. A stable sorting algorithm maintains the relative order of equal elements.

Bubble sort is stable because we only swap when `arr[j] > arr[j+1]` (strict greater than), not `>=`. Equal elements are never swapped, so their relative order is preserved.

Q5. What is the time complexity of parallel bubble sort?

Sequential bubble sort: $O(n^2)$ — n passes, each $O(n)$.

Parallel bubble sort (with unlimited processors): $O(n)$ — still needs n passes but each pass takes $O(1)$ parallel time.

Important: In Python with ThreadPoolExecutor (GIL + thread overhead), the parallel version may actually be slower than sequential for small arrays due to thread creation cost.

— ADVANCED QUESTIONS —

Q6. Why is bubble sort not used in practice? What is used instead?

Bubble sort is $O(n^2)$ — very slow for large datasets. In practice: — Python's built-in sort: Timsort — $O(n \log n)$, stable. — Merge sort: $O(n \log n)$, stable, parallelizable. — Quick sort: $O(n \log n)$ average, not stable, in-place. — Heap sort: $O(n \log n)$, not stable, in-place.

Bubble sort is used only for educational purposes to understand comparison-based sorting.

Q7. What does `time.time()` measure? How is execution time calculated?

`time.time()` returns the current time as a float (seconds since Unix epoch — Jan 1, 1970).

```
start_time = time.time()
# ... sorting code ...
end_time = time.time()
elapsed_ms = (end_time - start_time) * 1000 # convert to ms
```

This measures wall-clock time (real elapsed time), not CPU time. Multiplying by 1000 converts seconds to milliseconds.

HPC PRACTICAL 4 — PARALLEL MERGE SORT

Aim: Implement Sequential and Parallel Merge Sort with timing comparison

File: merge.py | Method: Divide & Conquer | ThreadPoolExecutor

— BASIC QUESTIONS —

Q1. Explain merge sort step by step.

Merge sort is a divide and conquer algorithm: 1. Divide: Split the array at the midpoint into left and right halves. 2. Conquer: Recursively sort each half. 3. Merge: Combine the two sorted halves into one sorted array.

```
Array: [38, 27, 43, 3]
Divide: [38, 27] | [43, 3]
Divide: [38][27] | [43][3]
Merge: [27,38] | [3,43]
Merge: [3, 27, 38, 43] <- final sorted
```

Q2. How is parallelism achieved in merge sort?

The left and right halves are completely independent — sorting one half doesn't affect the other. So they can be sorted simultaneously using two threads:

```
with ThreadPoolExecutor() as executor:
    f1 = executor.submit(parallel_merge_sort, arr, left, mid)
    f2 = executor.submit(parallel_merge_sort, arr, mid+1, right)
# Both threads run concurrently
# The 'with' block exits only after both complete
merge(arr, left, mid, right) # called after both done
```

Q3. What is the time and space complexity of merge sort?

Time Complexity: $O(n \log n)$ in ALL cases (best, average, worst). There are $\log n$ levels of recursion, each level does $O(n)$ work in the merge step.

Space Complexity: $O(n)$ — requires auxiliary arrays L and R during the merge step. Not in-place unlike bubble sort.

Parallel time with unlimited processors: $O(\log^2 n)$.

Q4. Explain the merge() function — how does it combine two sorted halves?

The merge function uses two pointers (i for left half, j for right half) and picks the smaller element at each step:

```
L = arr[left:mid+1] # copy of left half
R = arr[mid+1:right+1] # copy of right half
i, j, k = 0, 0, left
while i < len(L) and j < len(R):
    if L[i] <= R[j]:
        arr[k] = L[i]; i += 1
    else:
        arr[k] = R[j]; j += 1
    k += 1
# Copy remaining elements from L or R
```

Q5. Is merge sort stable? Compare with quick sort.

Merge sort: Stable, $O(n \log n)$ all cases, $O(n)$ space, parallelizable. Quick sort: Not stable, $O(n \log n)$ average but $O(n^2)$ worst case, $O(\log n)$ space, in-place.

Merge sort is stable because it uses $L[i] \leq R[j]$ — equal elements from the left array are placed first, preserving their relative order.

Merge sort is preferred when: stability is needed, working with linked lists, external sorting (large files), or parallel environments.

— ADVANCED QUESTIONS —

Q6. Why does the parallel version sometimes run slower than sequential?

Several reasons: 1. Thread creation overhead — creating/destroying threads takes time. For small arrays, this overhead exceeds the benefit. 2. Python's GIL — prevents true parallel CPU execution. 3. Context switching — OS switching between threads adds overhead. 4. Memory contention — both threads writing to the same array can cause cache misses.

Tip: For truly parallel merge sort, use multiprocessing.Pool or a threshold below which sequential sort is used.

Q7. What is divide and conquer? Name other algorithms using it.

Divide and conquer breaks a problem into smaller subproblems, solves each recursively, and combines the results. Three steps: Divide → Conquer → Combine.

Other D&C algorithms: Quick sort (divide around pivot), Binary search (halve search space), Strassen's matrix multiplication, Fast Fourier Transform (FFT), Closest pair of points.

HPC PRACTICAL 5 — PARALLEL REDUCTION

Aim: Compute Sum, Min, Max, Average in parallel using ThreadPoolExecutor

File: reduction.py | Method: Parallel Futures | Read-only operations

— BASIC QUESTIONS —

Q1. What is reduction in HPC?

Reduction is combining a large set of values into a single scalar result using an associative operation (sum, min, max, product, AND, OR).

In HPC, reduction is parallelized by: 1. Splitting the array into chunks. 2. Computing partial results on each chunk in parallel. 3. Combining partial results into the final answer.

Example: Sum of [1,2,3,4,5,6,7,8] Thread 1: $1+2+3+4 = 10$, Thread 2: $5+6+7+8 = 26$, Final: $10+26 = 36$

Q2. Explain how the code computes all 4 operations in parallel.

Four independent functions are submitted to the thread pool simultaneously. Each function operates on the same array but computes a different result:

```
with ThreadPoolExecutor() as executor:
    future_sum = executor.submit(calculate_sum, arr) # Thread 1
    future_min = executor.submit(calculate_min, arr) # Thread 2
    future_max = executor.submit(calculate_max, arr) # Thread 3
    future_avg = executor.submit(calculate_average, arr) # Thread 4

total_sum = future_sum.result() # blocks until Thread 1 done
```

All four futures are created BEFORE any .result() is called — this ensures all four tasks start in parallel.

Q3. What is a Future? How does it work?

A Future is an object that represents the result of an asynchronous computation that may not be complete yet.

`executor.submit(fn, args)` — returns a Future immediately without waiting. `future.result()` — blocks the current thread until the result is ready. `future.done()` — returns True if computation is complete.

Tip: Think of it like ordering food — you get a ticket (Future) immediately, and collect the food (result) when it's ready.

Q4. Are the four reduction operations truly independent? Any data hazard?

Yes, all four are read-only operations on the array — none of them modify `arr`. Multiple threads can safely read the same data simultaneously without any data hazard.

This is called a read-read scenario — no synchronization is needed when all threads only read shared data. Data hazards only occur when at least one thread writes.

Q5. What are the real-world applications of parallel reduction?

1. MapReduce (Hadoop, Spark) — processing terabytes of data across clusters. 2. GPU computing — CUDA reduction for computing statistics on large datasets. 3. Database aggregation — COUNT, SUM, AVG, MIN, MAX on millions of rows. 4. Signal processing — computing energy, RMS value of signals. 5. Machine learning — gradient aggregation in distributed training. 6. Scientific computing — computing norms, dot products on large vectors.

— ADVANCED QUESTIONS —

Q6. What is the difference between multiprocessing and multithreading in Python?

Multithreading: Multiple threads share same memory space. GIL prevents true CPU parallelism. Good for I/O-bound tasks (network, file I/O).

Multiprocessing: Multiple separate processes, each with own memory and Python interpreter. No GIL — true CPU parallelism. Good for CPU-bound tasks like numerical computation.

```
from multiprocessing import Pool
with Pool(4) as p:
    result = p.map(calculate_sum, chunks)
```

Tip: For real HPC reduction, multiprocessing.Pool should be used instead of ThreadPoolExecutor.

Q7. What is the time complexity of parallel reduction?

Sequential reduction: $O(n)$ — scan all n elements once.

Parallel reduction (tree-based with p processors): — $O(n/p)$ to compute partial sums on each processor. — $O(\log p)$ to combine partial results in tree fashion. — Total: $O(n/p + \log p)$

This code specifically: $O(n)$ per thread (Python's built-in sum/min/max are $O(n)$), but all 4 run simultaneously — wall time is approximately $O(n)$ instead of $O(4n)$.

DL PRACTICAL 1 — BOSTON HOUSING PRICE PREDICTION

Aim: Predict house prices using Deep Neural Network (Regression)

File: DL1_M.pdf | Dataset: Boston Housing (506 samples, 13 features) | Accuracy: 86.9% R^2

— BASIC QUESTIONS —

Q3. What is the aim of this practical?

To predict house prices in Boston using a Deep Neural Network (DNN). Given 13 input features (crime rate, number of rooms, tax rate, etc.), the model predicts the median house value (medv) in \$1000s.

This is a regression problem — the output is a continuous number, not a class label. The model learns the relationship between house features and price from historical data.

Q2. What is the dataset used? Describe its key features.

Boston Housing Dataset — 506 samples, 13 features, 1 target (medv). No missing values. All numerical features.

Key features: — crim: crime rate per town — rm: average rooms per house (highest +ve correlation: +0.695) — lstat: % lower-status population (strongest -ve correlation: -0.738) — ptratio: pupil-teacher ratio — nox: nitric oxide concentration — tax: property tax rate — medv: median home value in \$1000s (TARGET)

Q3. What is StandardScaler? Why is normalization needed?

StandardScaler applies Z-score normalization: subtracts mean and divides by standard deviation. Result: each feature has mean=0, std=1.

```
scaler = StandardScaler()
scaler.fit(X_train) # compute mean/std from training data ONLY
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test) # use same mean/std
```

Important: Always fit only on X_train, never on X_test — to avoid data leakage (test data influencing the scaler).

Without normalization: features with large ranges (tax: 100-700) dominate gradient updates, making training slow and unstable.

Q4. Explain the model architecture used.

```
Input: 13 features
Dense(128, relu, input_shape=(13,)) -> 1792 params
Dense(64, relu) -> 8256 params
Dense(1, linear) -> 65 params
Total: 10,113 parameters
```

relu: introduces non-linearity, avoids vanishing gradient, returns $\max(0, x)$. linear output: no activation on final layer — needed for regression to output any real value (not bounded 0-1). Softmax/sigmoid would prevent predicting values like 25 or 50.

Q5. What is MSE loss? Why not use accuracy as metric?

MSE (Mean Squared Error) = average of squared differences between predictions and actual values.
Formula: $MSE = (1/n) \times \sum (y_{\text{pred}} - y_{\text{actual}})^2$

Accuracy measures correct class predictions — meaningless for regression. If predicted price = 24.5 and actual = 25.0, it's not 'wrong', it's close. MSE and MAE quantify this closeness numerically.

MAE (metric used): average of $|y_{\text{pred}} - y_{\text{actual}}|$ — easier to interpret (average error in \$1000s).
Test MAE = 2.64 means predictions off by \$2,640 on average.

— ADVANCED QUESTIONS —

Q6. What is R^2 score? How do you interpret the 86.9% result?

R^2 (R-squared) = Coefficient of Determination. Formula: $R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$ SS_{res} = sum of squared residuals (prediction errors) SS_{tot} = total variance in the data

$R^2 = 0.869$ means: The model explains 86.9% of the variance in house prices. The remaining 13.1% is due to factors not captured by the 13 features (interior quality, year built, etc.). $R^2=1.0$: perfect. $R^2=0$: no better than mean. $R^2<0$: worse than mean.

Q7. What is Adam optimizer? How does it work?

Adam (Adaptive Moment Estimation) combines two improvements: 1. Momentum: uses exponentially weighted moving average of past gradients (m_t) — overcomes local minima. 2. RMSprop: uses moving average of squared gradients (v_t) — adapts learning rate per parameter.

Result: each parameter gets its own adaptive learning rate. Parameters with large gradients get smaller updates; sparse parameters get larger updates. Faster and more stable than plain SGD.

Q8. What is train_test_split? What does test_size=0.25 mean?

train_test_split randomly divides the dataset into training and testing sets. test_size=0.25 means 25% of data (approx. 127 samples) is used for testing, 75% (approx. 379 samples) for training.

Tip: Never train and evaluate on the same data — the model would just memorize the answers. The test set simulates unseen real-world data.

DL PRACTICAL 2 — IMDB MOVIE SENTIMENT ANALYSIS

Aim: Binary classification of movie reviews as positive or negative

File: DL2_M.pdf | Dataset: IMDB (25K train, 25K test) | Accuracy: 86.56%

— BASIC QUESTIONS —

Q1. What is the aim of this practical?

To classify IMDB movie reviews as positive (1) or negative (0) — a binary text classification task. Given a review text, the model predicts the sentiment.

Real-world applications: product reviews, social media monitoring, customer feedback analysis, brand reputation management.

Q2. What is the IMDB dataset? How is it preprocessed?

50,000 movie reviews — 25,000 train, 25,000 test. Balanced: 12,500 positive + 12,500 negative in each split.

Preprocessing: 1. Only top `max_words=10,000` most frequent words are kept — reduces vocabulary size. 2. Reviews are already encoded as integer sequences (word -> index). 3. `pad_sequences` pads shorter reviews and truncates longer ones to `max_sequence_length=500`.

```
x_train = keras.preprocessing.sequence.pad_sequences(x_train, maxlen=500)
```

Q3. What is an Embedding layer? Why is it used?

An Embedding layer converts integer word indices into dense vector representations (word embeddings). `Embedding(10000, 100)`: vocabulary of 10,000 words, each as a 100-dimensional vector.

Why not one-hot encoding? One-hot vectors are sparse (10,000 dimensions, mostly zeros) and don't capture word similarity. Embeddings are dense (100 dims) and learn that 'good' and 'great' are semantically similar — they end up with similar vectors. The embedding layer is trained with the model.

Q4. What is GlobalMaxPooling1D? Why is it used after Embedding?

After `Embedding(10000,100)`, output is 3D: (batch_size, 500, 100) — 500 timesteps, each 100-dimensional.

`GlobalMaxPooling1D` takes the maximum value across all 500 timesteps for each of the 100 dimensions → output: (batch_size, 100). It extracts the most prominent feature from the entire sequence, making the model invariant to where a word appears.

Q5. What is Dropout? Why is it used here?

`Dropout(0.5)` randomly sets 50% of neurons to zero during each training step. This prevents the model from relying too heavily on any single neuron — it must learn redundant representations.

In DL-2, the model reached 100% training accuracy but only 86.56% test accuracy — classic overfitting. Dropout helps reduce this gap. During inference (prediction), dropout is disabled and all neurons are active.

Tip: Analogy: 'Dropout is like making the team practice with random players absent — forces every player to be capable, making the team perform better overall.'

— ADVANCED QUESTIONS —

Q6. What is binary crossentropy loss? When is it used?

Binary Crossentropy = $-[y \times \log(p) + (1-y) \times \log(1-p)]$ where y = true label (0 or 1), p = predicted probability.

Used when there are exactly 2 classes and the output is a probability (sigmoid activation outputs 0 to 1). The log function heavily penalizes confident wrong predictions.

Q7. Explain the confusion matrix results from this practical.

The confusion matrix for 25,000 test reviews: — True Negative (10,897): Negative reviews correctly predicted as negative. — True Positive (10,744): Positive reviews correctly predicted as positive. — False Positive (1,603): Negative reviews wrongly predicted as positive (Type I error). — False Negative (1,756): Positive reviews wrongly predicted as negative (Type II error).

Precision (class 0) = 0.86, Recall = 0.87, F1 = 0.87. Overall accuracy = 87%.

Q8. What is overfitting? How to detect and prevent it?

Overfitting: Model performs very well on training data but poorly on new/test data. It memorizes rather than generalizes.

Detection: Training accuracy much higher than validation accuracy. In DL-2: train acc = 100%, val acc = 86.56%. Also, val_loss increases while train_loss decreases.

Prevention techniques: 1. Dropout — randomly disable neurons (used here). 2. Early stopping — stop training when val_loss starts increasing. 3. More training data. 4. L1/L2 regularization — penalize large weights. 5. Reduce model complexity — fewer layers/neurons.

DL PRACTICAL 3 — FASHION MNIST CNN CLASSIFIER

Aim: Classify fashion clothing images into 10 categories using CNN

File: DL3_M.pdf | Dataset: Fashion MNIST (60K train, 10K test) | Accuracy: 91.42%

— BASIC QUESTIONS —

Q1. What is a CNN? How is it different from a regular ANN?

CNN (Convolutional Neural Network) uses convolutional filters that scan over the input to detect local patterns (edges, curves, textures).

ANN (fully connected): Every neuron connects to every neuron in the next layer. For a 28x28 image = 784 inputs x 128 neurons = 100,352 parameters just in the first layer.

CNN: A 3x3 filter has only 9 weights (+1 bias), shared across the entire image — drastically fewer parameters. Also translation invariant — detects a feature regardless of where it appears in the image.

Q2. What does Conv2D(32, 3x3) mean? Explain each parameter.

`Conv2D(32, kernel_size=(3,3), activation='relu', input_shape=(28,28,1))`

32: number of filters (feature detectors). Each filter learns to detect a different pattern (horizontal edges, vertical edges, curves, etc.). (3,3): each filter is 3x3 pixels in size. The filter slides over the entire image with stride=1 (default). Output size: $(28-3+1) \times (28-3+1) \times 32 = 26 \times 26 \times 32$. Parameters: 32 filters x (3x3x1 weights + 1 bias) = 320 parameters.

Q3. What is MaxPooling2D? Why is it used?

`MaxPooling2D(pool_size=(2,2))` takes the maximum value in each 2x2 region, halving the spatial dimensions: 26x26 -> 13x13.

Why used: 1. Reduces computation — fewer values to process. 2. Makes model translation-invariant — a feature shifted slightly still gets detected. 3. Retains the most prominent features detected by the conv layer. 4. Reduces overfitting by discarding less important information.

Q4. What is the Flatten layer? Why is it needed?

After MaxPooling, the output is a 3D tensor: 13x13x32 = 5,408 values. Dense layers require a 1D input — `Flatten()` reshapes this 3D tensor into a 1D vector of 5,408 values.

Without Flatten, you cannot connect the convolutional layers to the fully connected Dense layers.

Q5. What is softmax activation? Why is it used in the output layer?

Softmax converts raw output scores (logits) into probabilities. For 10 classes, it outputs 10 values that sum to exactly 1.0.

Formula: $\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum(\exp(z_j))}$ for all j

Used in the output because this is a 10-class classification problem — we want the probability of each class. The class with the highest probability is the prediction.

Sigmoid (for 2-class binary) vs Softmax (for 3+ class multiclass): Use sigmoid for DL-2 (positive/negative), use softmax for DL-3 (10 clothing categories).

— ADVANCED QUESTIONS —

Q6. Explain the complete model architecture and total parameters.

```
Input: (28, 28, 1)
-> Conv2D(32, 3x3, relu) Output: (26,26,32) Params: 320
-> MaxPooling2D(2x2) Output: (13,13,32) Params: 0
-> Flatten() Output: (5408,) Params: 0
-> Dense(128, relu) Output: (128,) Params: 692,352
-> Dense(10, softmax) Output: (10,) Params: 1,290
Total: 693,962 params (2.65 MB)
```

Dense(128) params = 5408 inputs x 128 neurons + 128 biases = 692,352. Dense(10) params = 128 inputs x 10 neurons + 10 biases = 1,290.

Q7. What are the 10 classes in Fashion MNIST?

0: T-shirt/top, 1: Trouser, 2: Pullover, 3: Dress, 4: Coat, 5: Sandal, 6: Shirt, 7: Sneaker, 8: Bag, 9: Ankle boot

Each image is 28x28 grayscale pixels. Pixel values scaled from 0-255 to 0-1 by dividing by 255.0.

Q8. What is sparse categorical crossentropy? When to use it vs categorical crossentropy?

Sparse Categorical Crossentropy: used when class labels are integers (0, 1, 2, ..., 9) — as in DL-3. Categorical Crossentropy: used when labels are one-hot encoded ([1,0,0,...], [0,1,0,...]).

Binary Crossentropy: 2-class problems with sigmoid output (DL-2). Sparse Categorical: multiclass with integer labels (DL-3). Categorical: multiclass with one-hot labels.

Important: from_logits=True in DL-3 tells TensorFlow to apply softmax internally. Since the model already has a softmax output layer, this effectively applies softmax twice — a bug in the practical code. In practice, use from_logits=False when the model already has softmax.

Q9. What is the training result? Why is there a gap between train and test accuracy?

Training (5 epochs): Epoch 1: 86.06% -> Epoch 5: 93.88% (train accuracy) Test accuracy: 91.42%, Test loss: 0.2526

The train accuracy (93.88%) is slightly higher than test accuracy (91.42%) — mild overfitting. With only 5 epochs and a relatively small model, it's well-controlled. The 2% gap is acceptable.

To improve: add more Conv2D layers, use data augmentation, add Batch Normalization, train for more epochs with early stopping.

Q10 What is relu activation? Why not use sigmoid in hidden layers?

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. Outputs x if positive, else 0.

Why not sigmoid in hidden layers: 1. Vanishing gradient problem — sigmoid saturates near 0 and 1, gradients become tiny, deep networks don't learn. 2. ReLU gradients don't saturate for positive inputs — faster training. 3. ReLU is computationally simpler (no exponential calculation).